

Práctica Nro. 6

Parámetros

Objetivo: Descubrir las técnicas existentes para pasaje de parámetros entre unidades y sus diferencias esenciales de acuerdo al lenguaje que lo implementa.

Ejercicio 1: Explique brevemente los siguientes conceptos

- **Parámetro:** mecanismo que permite comunicar información entre una rutina llamadora y una rutina llamada. Es la forma más clara y recomendada de compartir datos entre subprogramas.
- **Parámetro real:** es el valor, variable o expresión que se pasa en la invocación a una rutina. Se especifica en la llamada. Son **(argumentos)** → **En invocación**. Ejemplo: *writeln (suma(5,4));*
- **Parámetro formal:** es la variable declarada en la definición de la rutina que recibe el valor del parámetro real. Variable local de la rutina. Son **(parámetros)** → **En declaración**. Ejemplo: *Function suma (a:integer): integer; {...función...}*
- **Ligadura posicional:** mecanismo mediante el cual los parámetros reales se asocian con los parámetros formales según el orden en que aparecen en la lista de parámetros.
- **Ligadura por palabra clave o nombre:** mecanismo en el que los parámetros reales se vinculan a los formales mediante sus nombres, por lo que pueden pasarse en un orden diferente al de la declaración.

Ejercicio 2: Unir los siguientes puntos según corresponda y de una definición y un ejemplo de cada par.

- Modo IN | Modo OUT | Modo IN/Out
- Resultado | Valor | Valor / Resultado | Nombre | Resultado de funciones | Valor Constante | Referencia

MODO	MECANISMO	DEFINICIÓN	EJEMPLO
IN	Valor	El valor del parámetro real se copia al parámetro formal al inicio de la ejecución. Luego no existe vínculo entre ellos. No modifica parámetro real.	<i>Proc P(a: integer); begin a := a + 1; end; i := 3; P(i);</i>
	Valor constante	El parámetro se utiliza solo como entrada y no puede modificarse dentro de la rutina.	<i>void a(const int x){ cout << x; }</i>
OUT	Resultado	El parámetro formal no recibe un valor inicial. Al finalizar la rutina, el valor del formal se copia al parámetro real.	<i>Proc P(res a: integer); begin a := 10; end; i := 3; P(i);</i>
	Resultado de funciones	Una función devuelve un valor que reemplaza a la llamada dentro de una expresión.	<i>int f(){ return 8;} x = f();</i>
IN/OUT	Valor/Resultado	Se copia el valor del parámetro real al formal al inicio y, al finalizar, se copia nuevamente el valor del formal al real. No hay vínculo durante la ejecución.	<i>Proc P(in-out a: integer); begin a := a + 3; end; i := 3; P(i);</i>
	Referencia	No se copia el valor sino una referencia a la variable original. Los cambios sobre el formal afectan inmediatamente al real.	<i>Proc P(var a: integer); begin a := a + 3; end; i := 3; P(i);</i>
	Nombre	Parámetro formal se reemplaza por parámetro real y se evalúa cada vez que se utiliza, en el entorno del llamador.	<i>P(a[m]);</i>

Ejercicio 3: Complete el siguiente cuadro según lo correspondiente a cada lenguaje:

- **ADA:** IN (por copia), OUT (por resultado), IN/OUT. Para tipos primitivos usa valor-resultado y para tipos compuestos referencia.
- **C:** Por valor. Si se desea referencia se utilizan punteros. También admite valor constante con *const*.
- **Ruby:** Por referencia a objetos.
- **Java:** Solo por valor. En objetos se copia la referencia al objeto, por lo que su comportamiento puede parecer por referencia.
- **Phyton:** Pasa objetos. Los inmutables se comportan como por valor; los mutables (listas, diccionarios, etc.) pueden modificarse sin hacer copia.

Ada es más seguro que Pascal, respecto al pasaje de parámetros en las funciones.

Explique por qué.

Ada es más seguro porque **en las funciones solo permite el pasaje por copia de valor**, evitando que los parámetros puedan modificarse mediante mecanismos que generen efectos colaterales. De esta forma, las funciones se comportan de manera más predecible y confiable.

Explique cómo maneja Ada los tipos de parámetros in-out de acuerdo al tipo de dato.

En Ada, los parámetros IN/OUT se implementan de forma diferente según el tipo de dato. Para tipos primitivos se utiliza pasaje por valor-resultado, mientras que para tipos compuestos o no primitivos se utiliza pasaje por referencia.

Ejercicio 4

¿Existió algún caso que no pudo realizarlo porque saltó algún tipo de error? Diga cuál y por qué.

El último caso (pasaje por resultado) tanto en cadena estática y dinámica, genera un error puesto que el parámetro formal no recibe ningún valor al entrar (es decir, llega sin inicializar) entonces se intenta leer un valor indefinido y eso genera un error.

¿Daré el mismo resultado si se trata de un lenguaje que sigue la cadena dinámica?

Justifique la respuesta realizando las pilas de activación

No dará el mismo resultado puesto que, con cadena estática, el procedimiento *Recibe* utiliza la variable *m* declarada en Main, y con cadena dinámica, *Recibe* utiliza la variable *m* declarada en Dos, porque es la primera que encuentra recorriendo la cadena de llamadas.

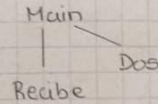
Ejercicio 4

```

Procedure Main;
var j, m, i: integer;
Procedure Recibe (x: integer; y: integer)
begin
  m := m + 1 + y;
  x := i + x + j;
  y := m - 1;
  write (x, y, i, j, m);
end;
Procedure Dos;
var m: integer;
begin
  m := 5;
  Recibe (x, j);
  write (i, j, m);
end;
begin
  m := 2;
  i := 1;
  j := 3;
  Dos;
  write (i, j, m);
end.

```

ÁRBOL DE ACTIVACIÓN SINTÁCTICO (depende de donde están declarados los procedimientos)



REGISTRO DE ACTIVACIÓN

*** Reg Act Main

```

*1 PR
j
m
i
Procedure Recibe
Procedure Dos
VR

```

*** Reg Act Dos

```

*2 PR
EE(*1)
ED(*1)
m
VR

```

*** Reg Act Recibe

```

PR
EE(*1)
ED(*2)

```

x

y

VR

} parametros

5) i → Paraje por referencia

Pila Estática

→ Imprime

*** Reg Act Main

*1 PR

j 3 5

m 2 6

i 1 5

Procedure Recibe

Procedure Dos

VR

*** Reg Act Dos

*2 PR

EE(*1)

ED(*1)

m 5

VR

*** Reg Act Recibe

*3 PR

EE(*1)

ED(*2)

x

y

VR

5 5 5 5 6 (Recibe)

5 5 5 (Dos)

5 5 6 (Main)

Imprime

(Recibe) 5 8 5 8 9

(Dos) 5 8 9

(Main) 5 8 2

Pila Dinámica

*** Reg Act Main

*1 PR

j 3 8

m 2

i 1 5

Procedure Recibe

Procedure Dos

VR

*** Reg Act Dos

*2 PR

EE(*1)

ED(*1)

m 5 9

VR

*** Reg Act Recibe

*3 PR

EE(*1)

ED(*2)

x

y

VR

Paso por referencia

NOTA

ii → Pasaje por Valor

*** RegAct Main

*1 PR

j: 3

m: 2 6

i: 1

Procedure Reabe

Procedure Dos

VR

*** RegAct Dos

*2 PR

EE(*1)

ED(*1)

m: 5

VR

*** RegAct Reabe

*3 PR

EE(*1)

ED(*2)

x: 1 5

y: 3 5

VR

→ Pila Estática

Imprime

55136 (Reabe)

135 (Dos)

136 (Main)

*** RegAct Main

*1 PR

j: 3

m: 2

i: 1

Procedure Reabe

Procedure Dos

VR

*** RegAct Dos

*2 PR

EE(*1)

ED(*1)

m: 5 9

VR

*** RegAct Reabe

*3 PR

EE(*1)

ED(*2)

x: 1 5

y: 3 5

VR

iii → Pasaje por Valor-Resultado (los cambios en el Procedure recién se pasan al Procedure que lo llamó una vez salgo)

Pila Estática

→ Imprime

Pila Dinámica

VR

VR

iii → Pasaje por Valor-Resultado (los cambios en el Procedure recién se pasan al Procedure que lo llamó una vez salgo)

Pila Estática

→ Imprime

Pila Dinámica

*** RegAct Main

*1 PR

j: 3 5

m: 2 6

i: 1 5

Procedure Reabe

Procedure Dos

VR

*** RegAct Dos

*2 PR

EE(*1)

ED(*1)

m: 5

VR

*** RegAct Reabe

*3 PR

EE(*1)

ED(*2)

x: 1 5

y: 3 5

VR

55136 (Reabe)

555 (Dos)

556 (Main)

Ⓢ Devuelve el valor resultado a Main cuando sale de Reabe (imprime antes de esto)

Imprime

58139 (Reabe)

589 (Dos)

582 (Main)

*** RegAct Main

*1 PR

j: 3 5

m: 2

i: 1 5

Procedure Reabe

Procedure Dos

VR

*** RegAct Dos

*2 PR

EE(*1)

ED(*1)

m: 5 9

VR

*** RegAct Reabe

*3 PR

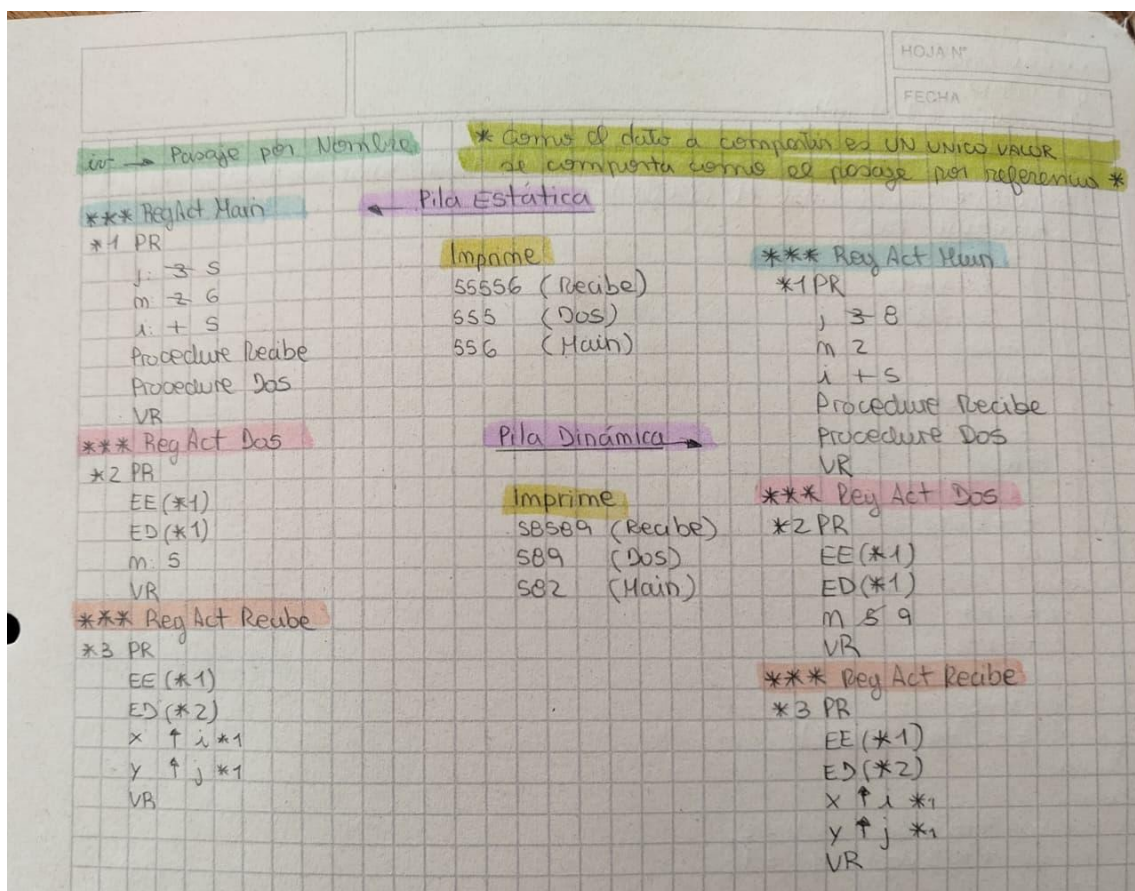
EE(*1)

ED(*2)

x: 1 5

y: 3 5

VR



Ejercicio 5: Suponiendo que se está ejecutando un programa con el siguiente registro de activación en memoria y se llama al procedimiento *rutina(iter,vec,a)*. Determine el tipo de parámetro que se deben utilizar en el llamado para que los resultados sean los siguientes:

a) (4,6,7), (4,6,7), 2, 2

- vec cambia de [3,5,6] a [4,6,7].
- vector también vale [4,6,7].
- a pasa de valer -1 a 2 y vit vale 2.

Como todos los cambios se reflejan afuera de la rutina, el tipo de parámetro es modo IN/OUT por referencia (en los tres casos).

b) (3,5,6), (4,6,7), 2, 2

- vec sigue valiendo [3,5,6] entonces no cambió.
- vector si cambió, vale [4,6,7].
- a pasa de valer -1 a 2 y vit vale 2.

Modo IN por valor para el parámetro vector (pues vec no cambió) y modo IN/OUT por referencia para los otros dos parámetros.

c) (3,5,6), (5,5,6), 0, -1

- *vec* sigue valiendo [3,5,6] entonces no cambió.
- *vector* si cambió, vale [5,5,6].
- *a* tampoco cambia.
- *vit* paso a valer 0 (o sea cambia en la rutina, pero no se ve el cambio afuera).

Modo IN por valor para todos los parámetros, puesto que los cambios se reflejan dentro de la rutina, pero fuera no.

Ejercicio 6: Indique con un ejemplo el comportamiento del parámetro por nombre (en el parámetro formal) para los siguientes casos de parámetros reales. ¿Qué sucede en cada caso?

- Un valor entero: se comporta exactamente igual que un pasaje por referencia.
- Una constante: es equivalente a por valor.
- Un elemento de un arreglo: puede cambiar el suscrito entre las distintas referencias.
- Una expresión: se evalúa cada vez.

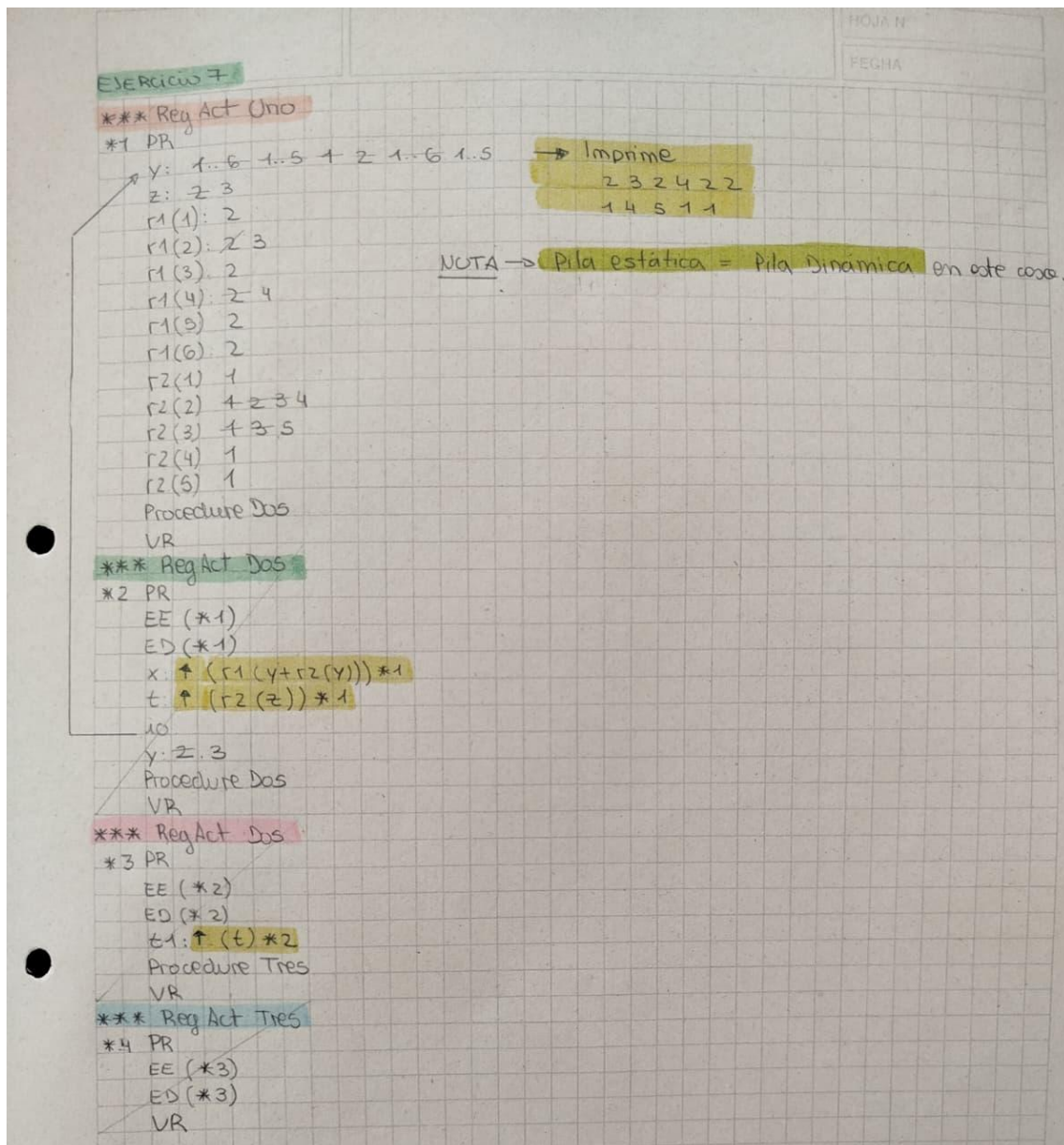
Ejercicio 7:

```

Procedure Uno;
  y, z: integer;
  r1: array[1..6] of integer;
  r2: array[1..5] of integer;
  Procedure Dos (nombre x, t: integer; var io: integer; valor-resultado y: integer);
    Procedure Dos (nombre t1: integer);
      Procedure Tres;
      begin
        y:=y+1;
        z:=z+1;
      end;
      begin
        t1:=t1+1;
        t:=t+1;
        Tres;
        t1:=t1+2;
        t:=t+2;
      end;
    begin
      x:=x+1;
      t:=t+1;
      io:=io+1;
      x:=x+2;
      if z=2 then Dos(t);
    end;
  begin
    for y:=1 to 6 do r1(y):=2;
    for y:=1 to 5 do r2(y):=1;
    z:=2;
    y:=1;
    Dos(r1(y+r2(y)), r2(z), y, z);
    for y:=1 to 6 do write(r1(y));
    for y:=1 to 5 do write(r2(y));
  end.

```

$$r1(y+r2(y)) = r1(1+r2(1)) = r1(2) = 2$$
 (memoria → recibe un elemento de un arreglo)
 Cada vez que aparece x, vuelvo a



Ejercicio 8: Indique las diferencias entre los pasajes de subprogramas como parámetros deep y shallow.

Ligadura Deep [profunda]: el ambiente es el del subprograma en donde se declara el subprograma usado como parámetro real. Se utiliza en los lenguajes con alcance estático y estructura de bloque. “Cuando un subprograma se pasa como parámetro, las referencias a variables libres se resuelven usando el ambiente existente en el momento en que el subprograma fue pasado como parámetro.”

Ligadura Shallow [superficial]: el ambiente de referencia es el del subprograma que tiene declarado el parámetro formal del subprograma. “Las referencias a variables libres se resuelven usando el ambiente del subprograma que contiene el parámetro formal al momento de la invocación.”

EJERCICIO 8

Program A

```
var x: integer;
var y: char;
```

Procedure B

```
begin
  var h: integer;
  h = 1 + x;
  write(y);
  C(D);
  write(y);
end;
```

Procedure C (Subrutina S)

```
begin
  var x: integer;
  var y: char;
  x = 3;
  y = 'b';
  x = S(x, y);
  y = 'j';
  write(x, y);
end;
```

Function D (j: integer, k: char);

```
begin
  j = j + x;
  k = y;
  write(k);
  return(j);
end;
```

```
begin
  x = 0;
  y = 'a';
  B();
  write(x, y);
end;
```

→ Shallow

*** Reg Act A

```
*1 PR
  x = 0;
  y = 'a';
  Procedure B;
  Procedure C;
  Function D;
  VR
```

*** Reg Act B

```
*2 PR
  EE (*1)
  ED (*1)
  h = 1;
  VR
```

*** Reg Act C

```
*3 PR
  EE (*1)
  ED (*2)
  x = 3;
  y = 'b';
  VR
```

*** Reg Act D

```
*4 PR
  EE (1)
  ED (3)
  j = 3;
  k = 'b';
  VR 6
```

Imprime

```
'a'
'b'
6'j'
'a'
0'a'
```

→ Deep

*** Reg Act A

```
*1 PR
  x = 0;
  y = 'a';
  Procedure B;
  Procedure C;
  Function D;
  VR
```

*** Reg Act B

```
*2 PR
  EE (*1)
  ED (*1)
  h = 1;
  VR
```

*** Reg Act C

```
*3 PR
  EE (*1)
  ED (*2)
  x = 3;
  y = 'b';
  VR
```

*** Reg Act D

```
*4 PR
  EE (*1)
  ED (*3)
  j = 3;
  k = 'b';
  VR 3
```

Imprime

```
'a'
'a'
3'j'
'a'
0'a'
```